

COMP20008 Elements of Data Processing



Lecture 10: Unsupervised: Hierarchical Clustering and Dimensionality Reduction

Semester 2 2026

Subject Coordinator: **Dr Ekaterina Vylomova**

Co-lecturer: **Dr Hanzhuo (Vivian) Ma**



THE UNIVERSITY OF
MELBOURNE

Reminders

- By **this FRIDAY 5PM:**
 - Assignment 1 code submission
 - Finalise your group for A2 and register it!

Announcements

- **More consultation sessions:**

- Student Hub (Melbourne Connect; Level 3):

- Monday 1–3pm

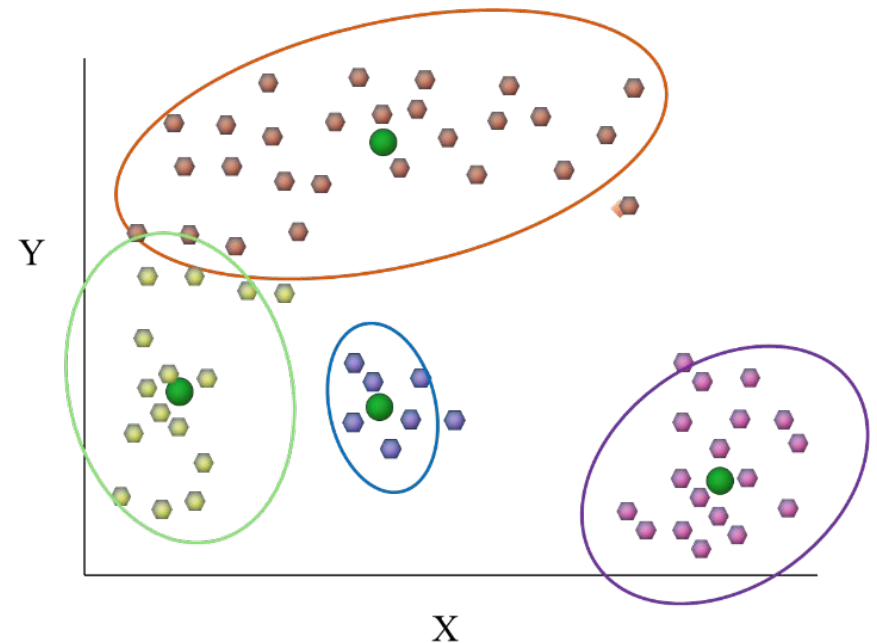
- Friday 1–3pm

- Hub: <https://cis.unimelb.edu.au/study/cis-student-hub>

- After Monday Lecture, i.e. 6pm, same theatre

Recap: Three desiderata for clusters

1. We must assign each object to exactly one cluster
2. We want similar objects in a cluster and different objects in different clusters
3. We want each cluster to be summarised by its **centroid** (the 'average' of all its objects)



Requires: – similarity measure
– adjusting the number of clusters
→ assessment the cluster quality

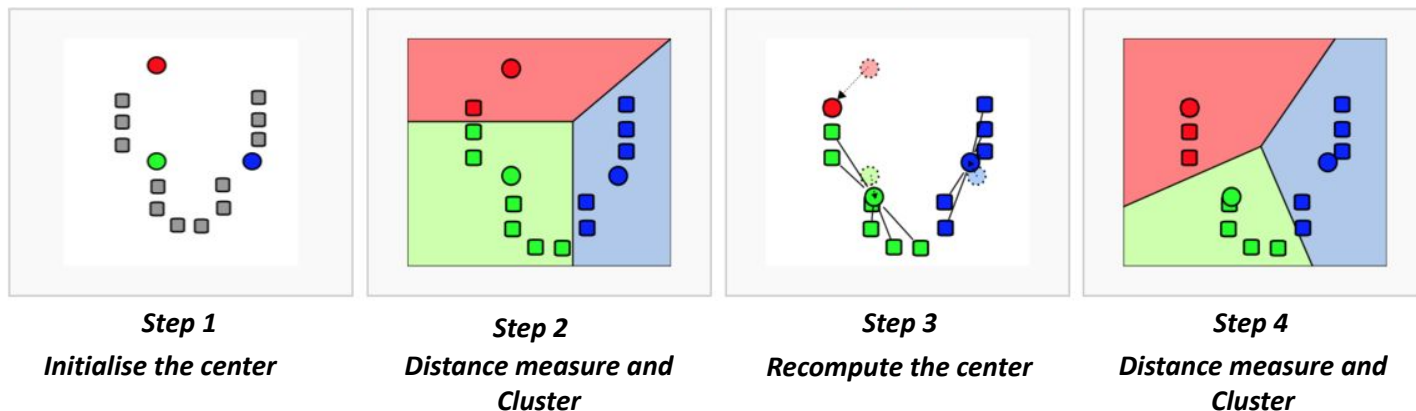
Recap: k-means

Partitioning clustering are clustering methods that require the analyst to specify **the number of clusters to be generated**.

how many clusters we want?

The best-known family of partitioning clustering algorithms is **k-means clustering**.

Demonstration of the standard K-means clustering algorithm



Recap: k-means

Partitioning clustering are clustering methods that require the analyst to specify **the number of clusters to be generated**.

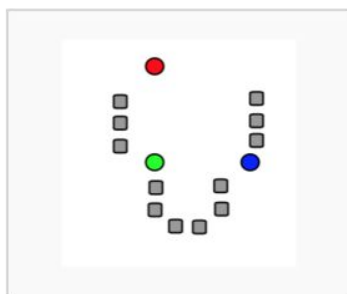
how many clusters we want?

The best-known family of partitioning clustering algorithms is **k-means clustering**.

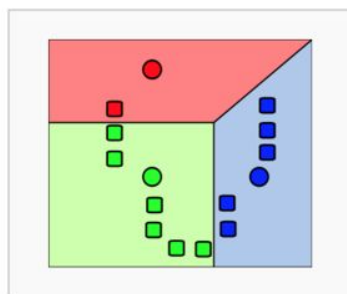
Assign

Update

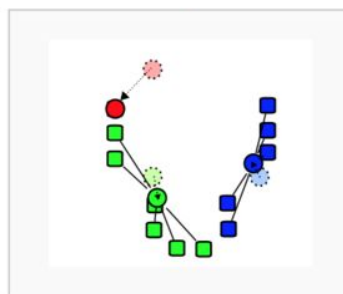
Demonstration of the standard K-means clustering algorithm



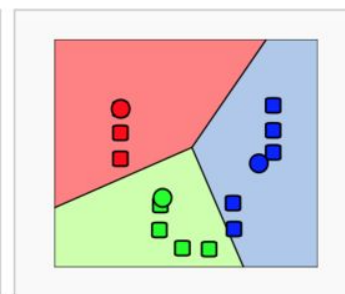
Step 1
Initialise the center



Step 2
Distance measure and Cluster



Step 3
Recompute the center



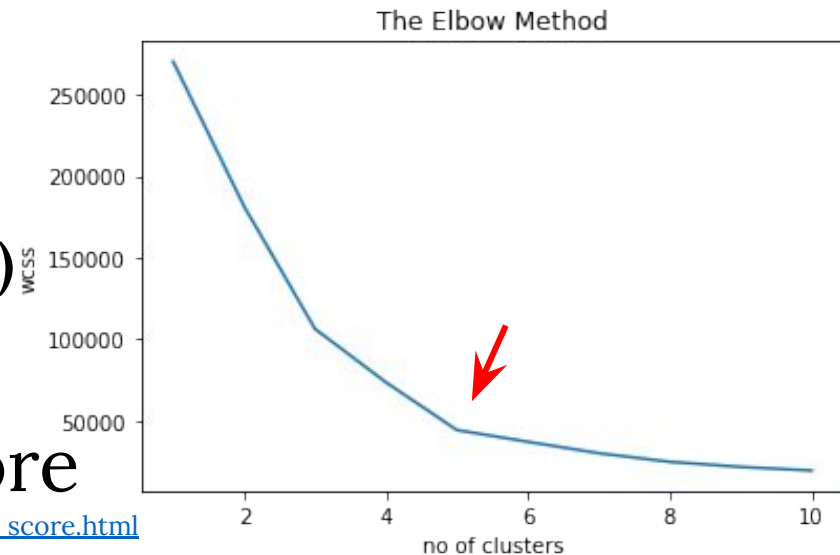
Step 4
Distance measure and Cluster

Recap: Finding k – the elbow method

- **‘Tightness’** measure: Within Cluster Sum of Squared distance (WCSS)

$$WCSS(k) = \sum_{c=1}^k \sum_{x_i \in c} \|x_i - \bar{x}_c\|^2, \quad \bar{x}_c \text{ is the centroid of cluster } c$$

- Sum of squared distances of data points to their closest cluster centre
- Called **SSE (Sum of Squared Error)**



Also common: Silhouette score

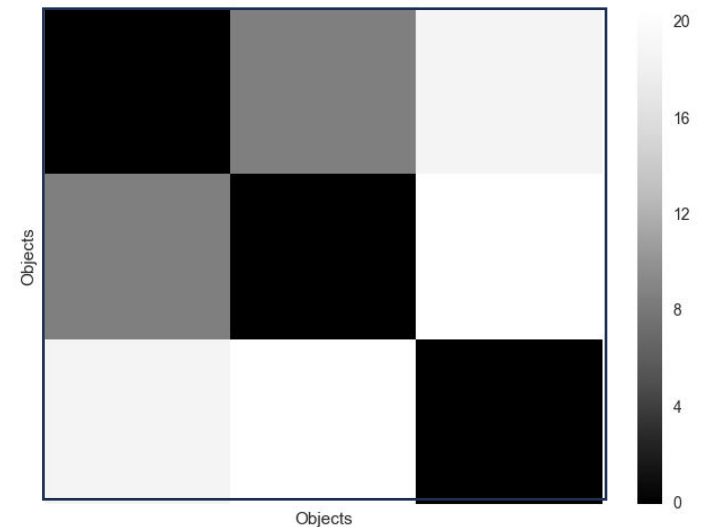
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html

<https://www.kaggle.com/abhishekyadav5/kmeans-clustering-with-elbow-method-and-sil>

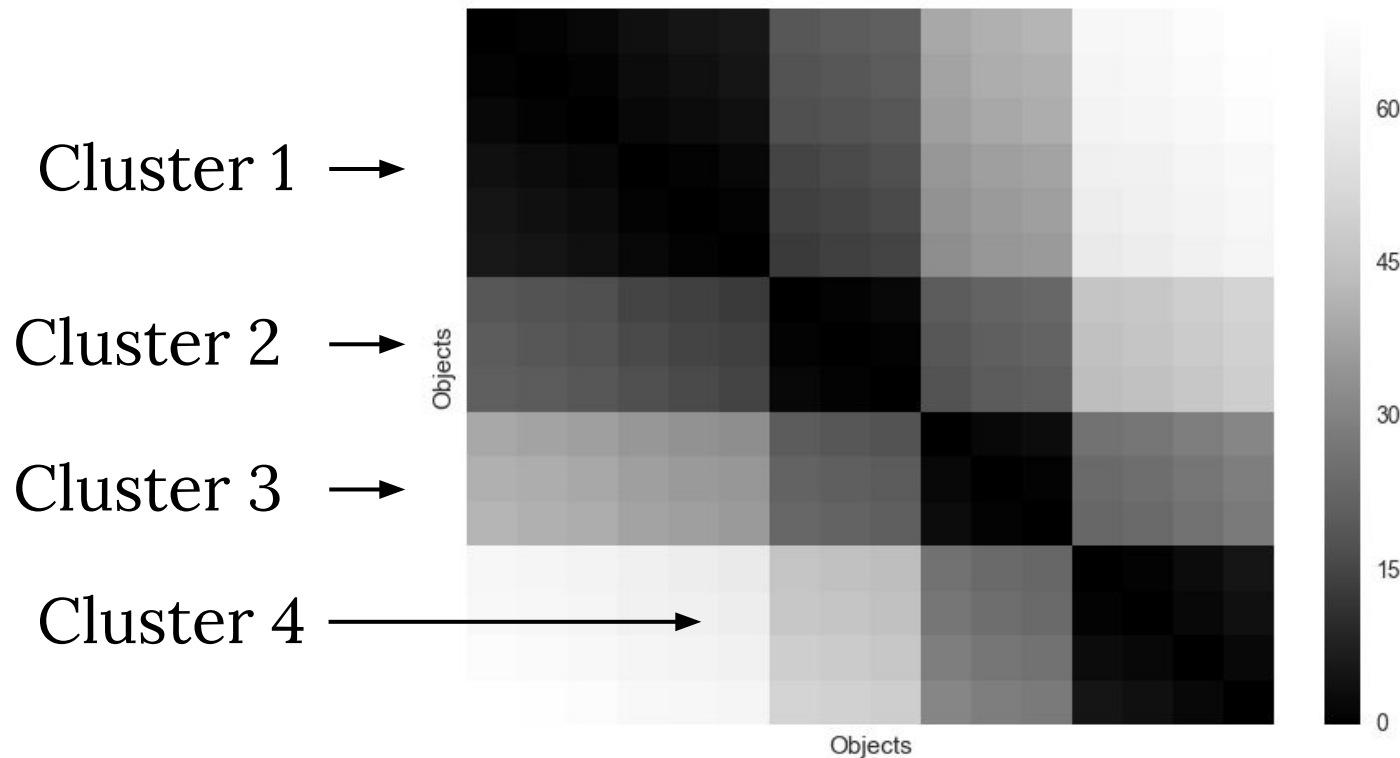
Recap: VAT & Visualising a dissimilarity matrix

Visualise a dissimilarity matrix as a **heatmap**, where the colour of each cell indicates that cell's value.

Object	1	2	3
1	0	8.7	18.7
2	8.7	0	20.6
3	18.7	20.6	0



Recap: VAT & Reordering the matrix reveals clusters



A better ordering of the 16 objects. **Nearby objects in the ordering are similar to each other, producing large dark blocks.**

Lecture outline

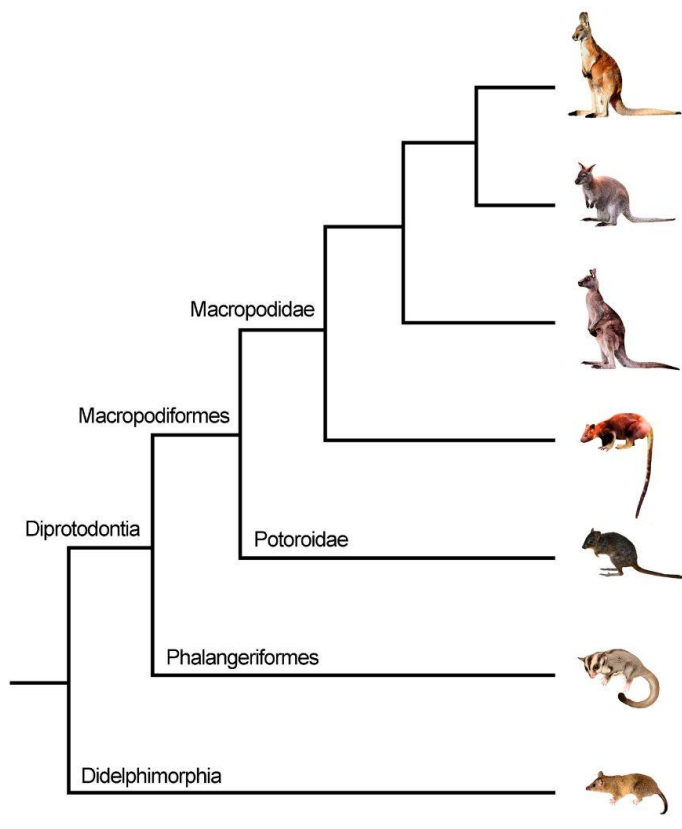
1. General introduction
2. k-Means clustering
3. Visualisation of clustering tendency (VAT)
4. Hierarchical clustering (Part 2)
5. Dimensionality reduction (Part 2)

Lecture outline

1. General introduction
2. k-Means clustering
3. Visualisation of clustering tendency (VAT)
4. Hierarchical clustering (Part 2)
5. Dimensionality reduction (Part 2)

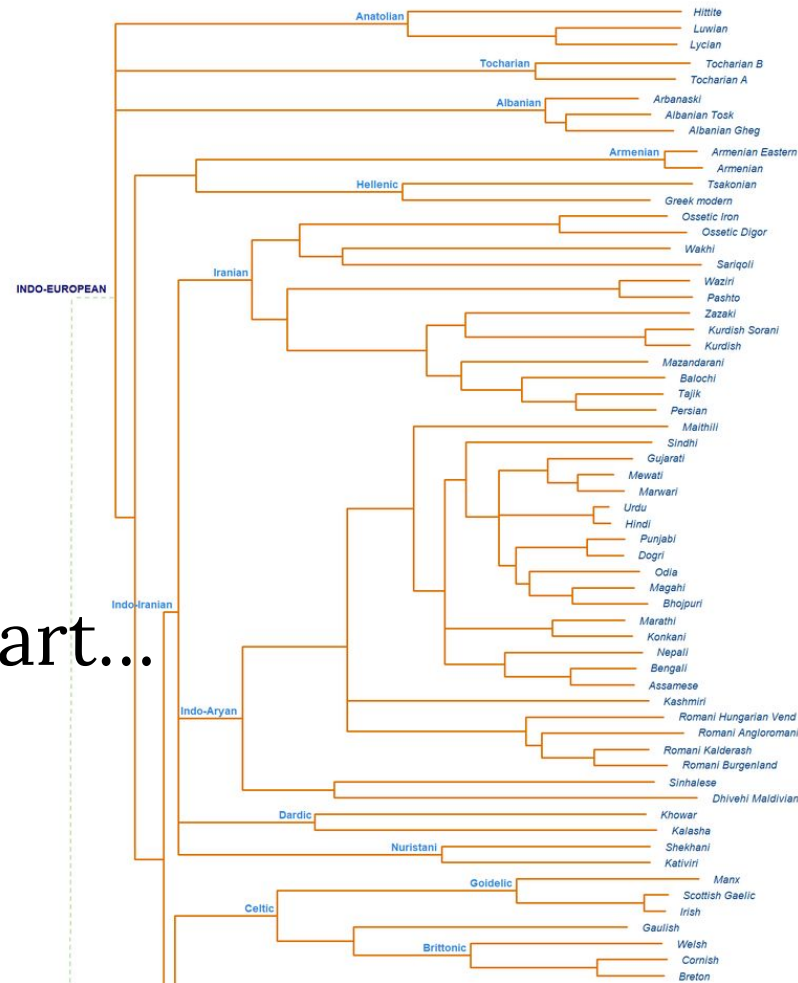
The world is highly
complex and hierarchical!

Taxonomies: Marsupials

		Species	Preferred gait	Majority of individuals are lateralised	Population lateralisation	Level of manual lateralisation
 Phylogenetic tree of Marsupials: - Diprotodontia - Macropodiformes - Macropodidae <i>M. (O.) rufus</i> Red kangaroo <i>M. (N.) rufogriseus</i> Red-necked wallaby <i>M. (M.) giganteus</i> Eastern grey kangaroo <i>D. goodfellowi</i> Goodfellow's tree kangaroo - Potoroidae <i>B. penicillata</i> Brush-tailed bettong - Phalangeriformes <i>P. breviceps</i> Sugar glider - Didelphimorphia <i>M. domestica</i> Grey short-tailed opossum		<i>M. (O.) rufus</i> Red kangaroo	Bipedal	Yes	Yes	5
		<i>M. (N.) rufogriseus</i> Red-necked wallaby	Bipedal	Yes	Yes except feeding in quadrupedal position	4
		<i>M. (M.) giganteus</i> Eastern grey kangaroo	Bipedal	Yes	Yes	5
		<i>D. goodfellowi</i> Goodfellow's tree kangaroo	Quadrupedal	Yes	No	2
		<i>B. penicillata</i> Brush-tailed bettong	Bipedal	Yes except support at the tripedal stance	Yes	5
		<i>P. breviceps</i> Sugar glider	Quadrupedal	No	No	2
		<i>M. domestica</i> Grey short-tailed opossum	Quadrupedal	No	No	2

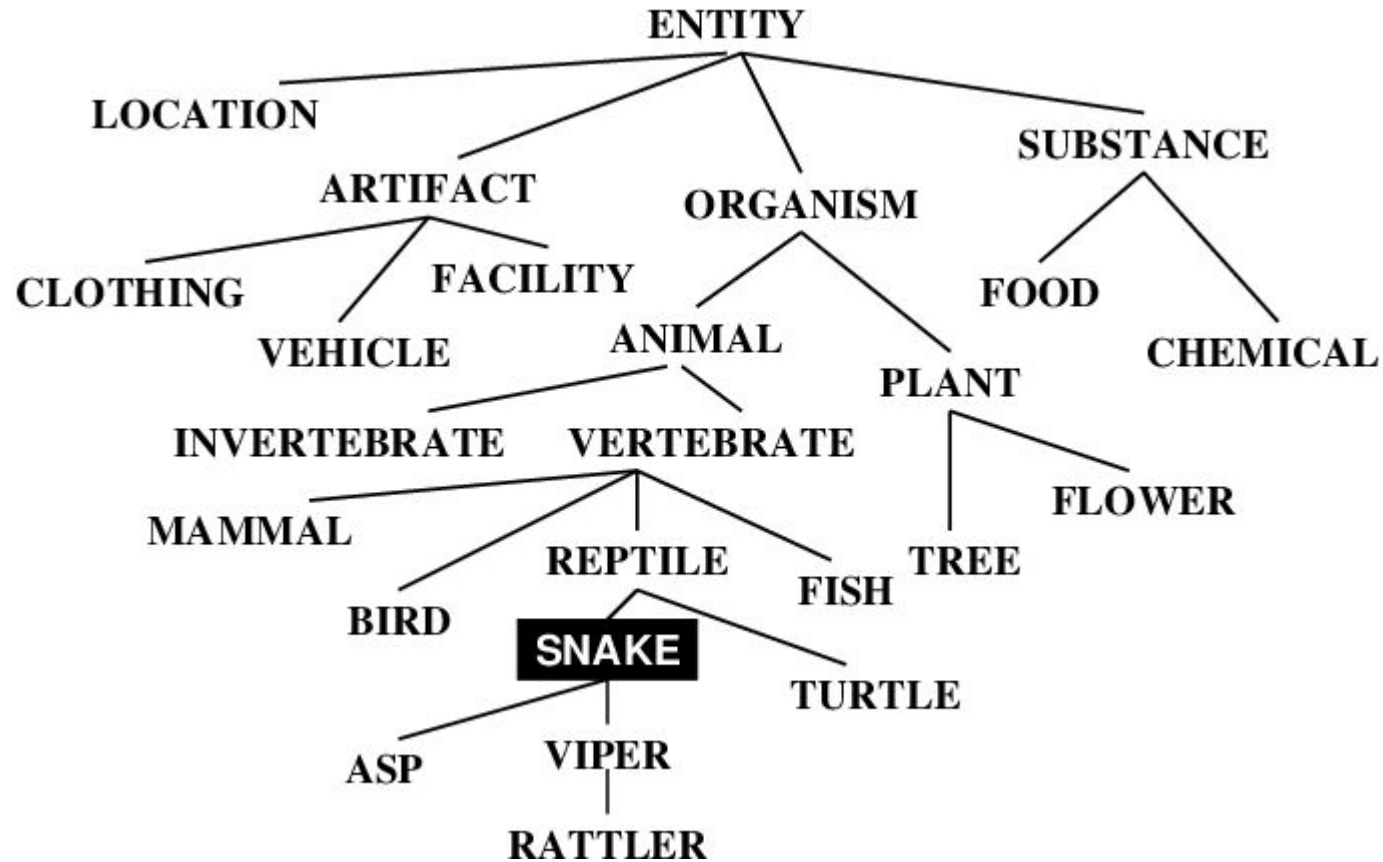
<https://www.sciencedirect.com/science/article/pii/S096098221500617X>

Phylogeny: Languages



Full: [http://www.elinguistics.net/Language Evolutionary Tree.html](http://www.elinguistics.net/Language_Evolutionary_Tree.html)

Ontologies: Wordnet



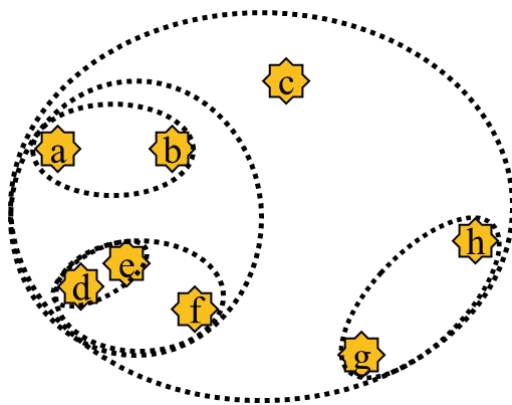
<https://wordnet.princeton.edu/>

multilevel hierarchy

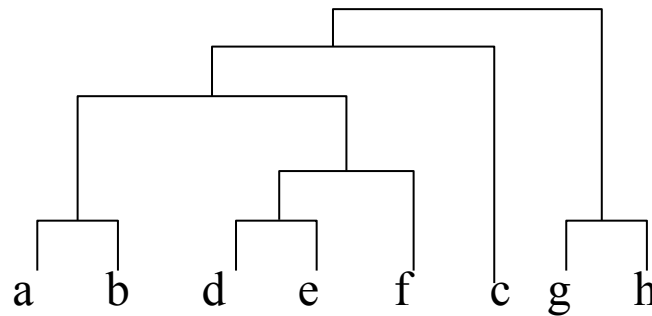


Hierarchical clustering

- Produces a set of **nested** clusters organized as a hierarchical tree
- Can be visualised as a **dendrogram**
 - A tree like diagram that records the sequences of merges (y-axis is distance)



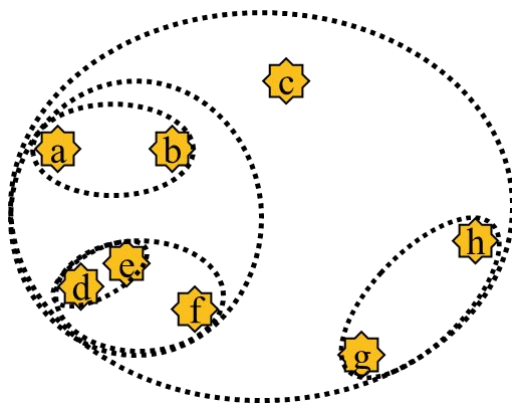
Nested Clusters



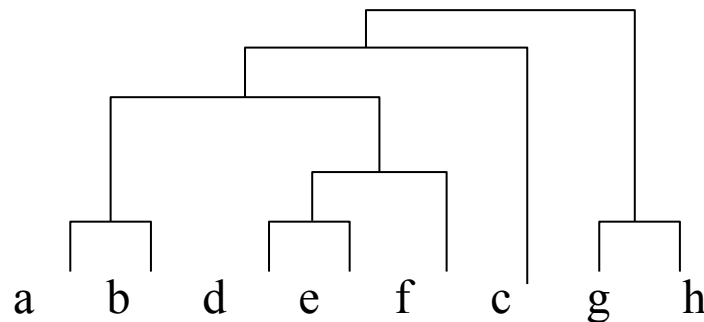
Dendrogram

Hierarchical clustering

- **Why use Hierarchical Clustering?**
 - **Do not have to assume any particular number of clusters**
 - Any desired number of clusters can be obtained by 'cutting' the dendrogram at the proper level
 - They may correspond to **meaningful taxonomies**
 - Example in biological sciences (e.g., animal kingdom, phylogeny reconstruction, ...)

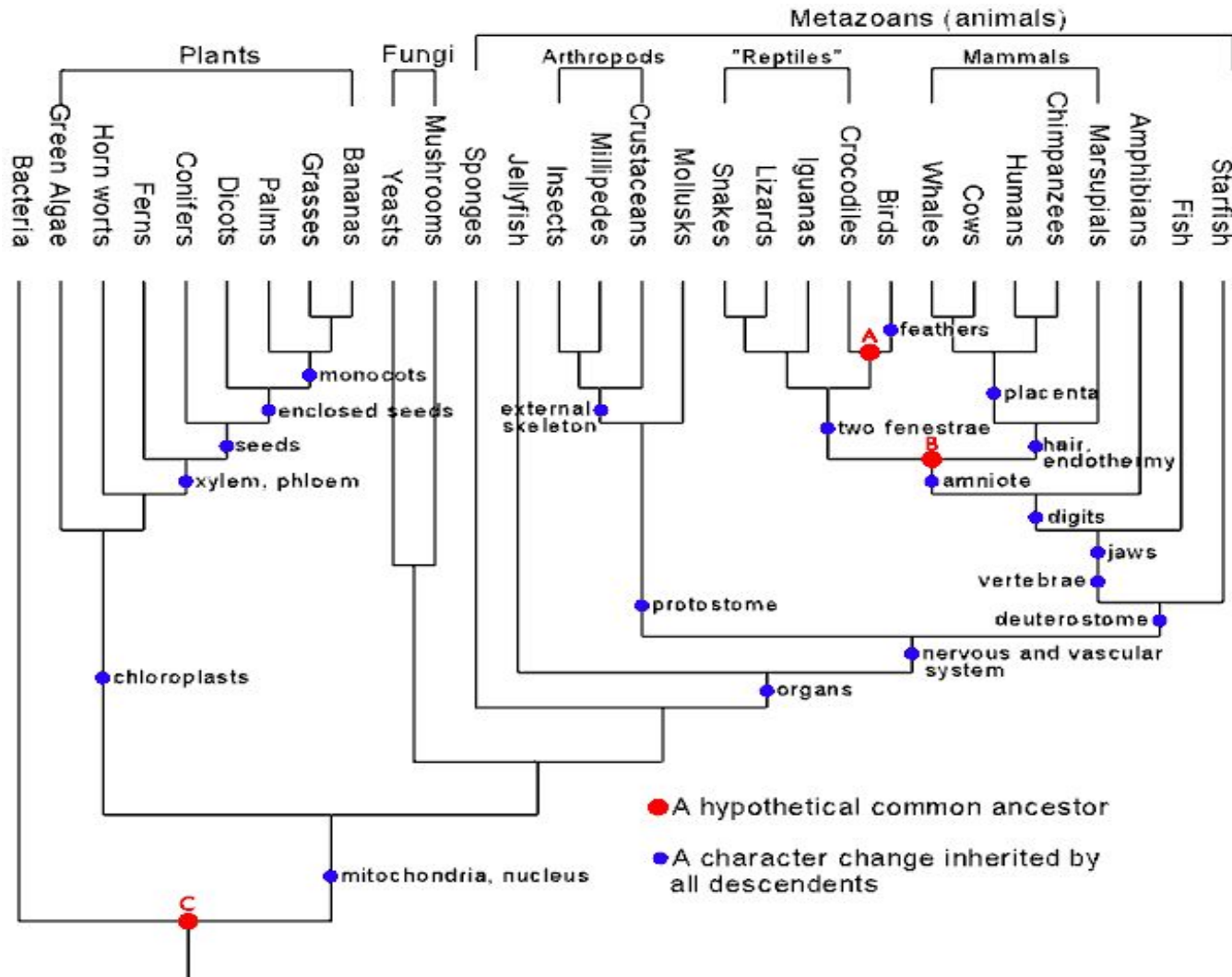


Nested Clusters



Dendrogram

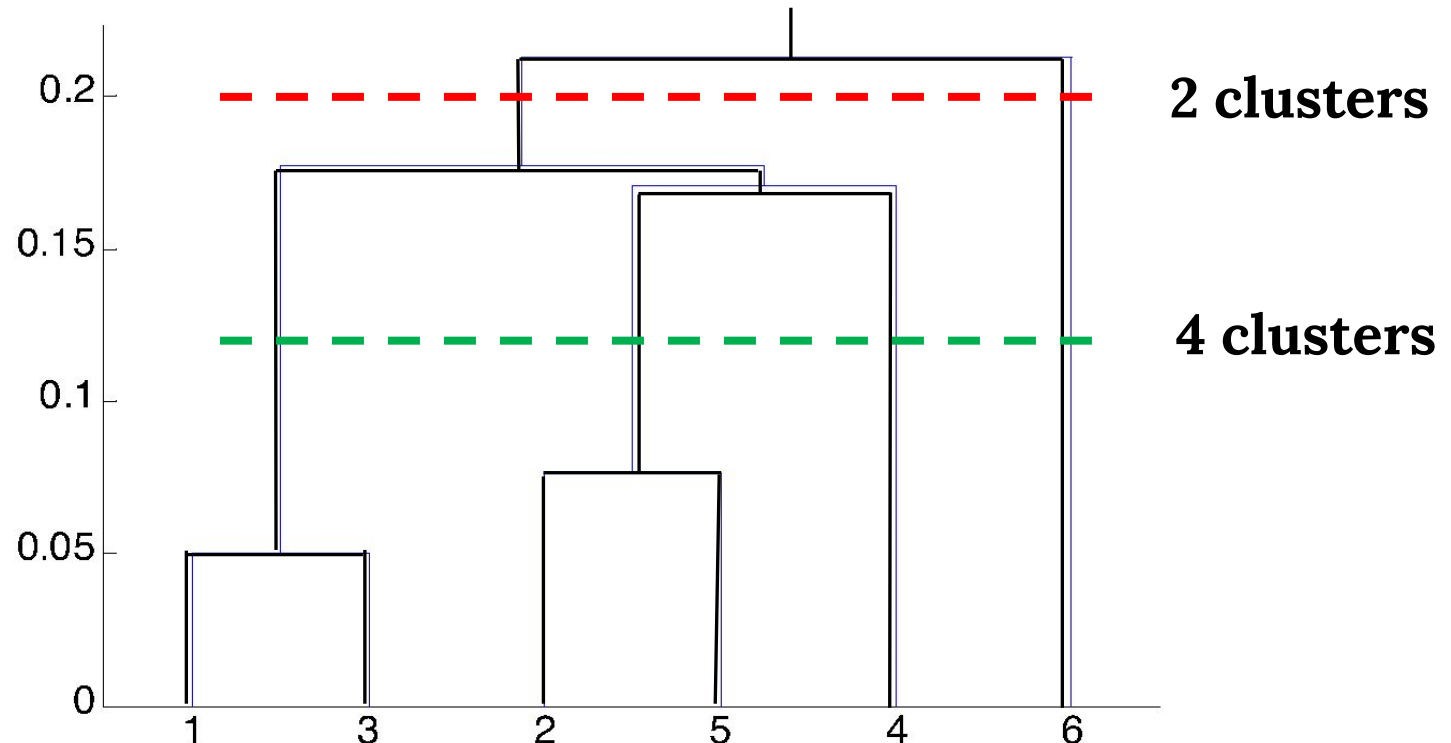
Example: Tree of life



How many clusters?

By 'cutting' the dendrogram

- If we cut high → fewer clusters
- If we cut low → more clusters

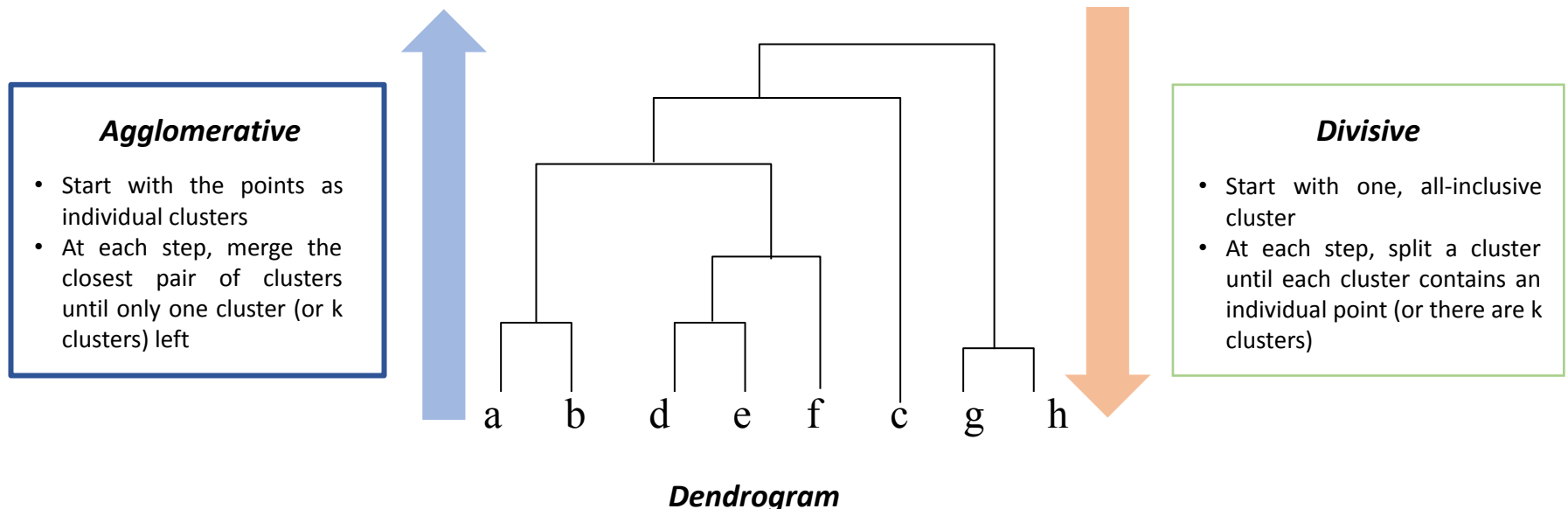


Hierarchical Clustering

Two main types of hierarchical clustering

1. **Agglomerative Clustering (bottom up)**
2. **Divisive Clustering (top down)**

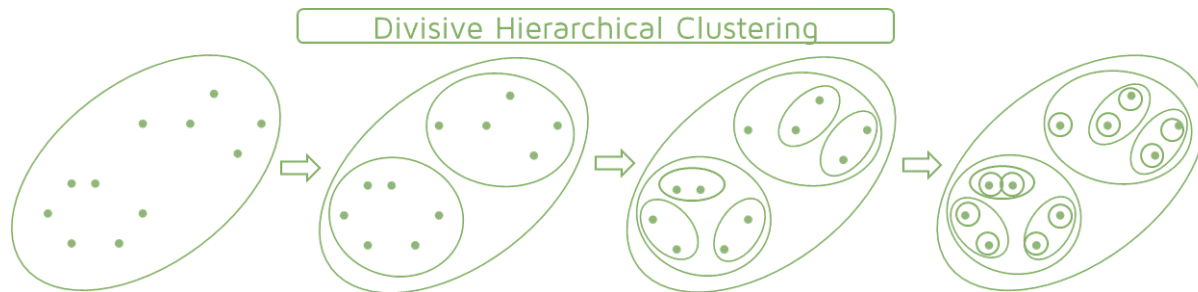
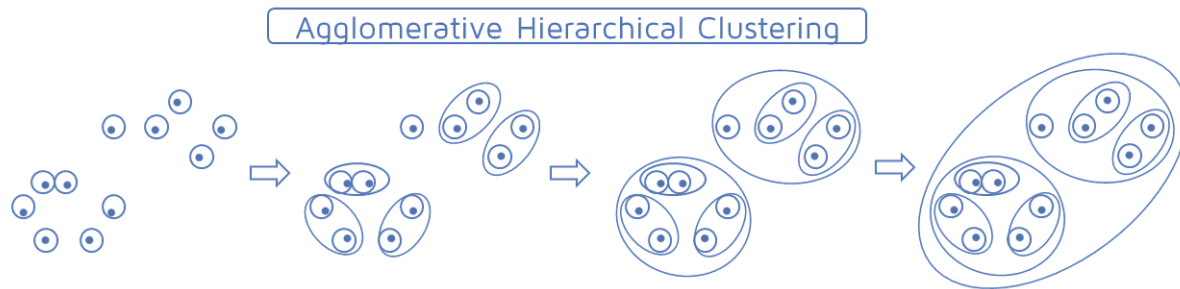
Traditional hierarchical algorithms use a similarity or distance matrix (Merge or split one cluster at a time)



Hierarchical Clustering

Two main types of hierarchical clustering

1. *Agglomerative Clustering (bottom up)*
2. *Divisive Clustering (top down)*

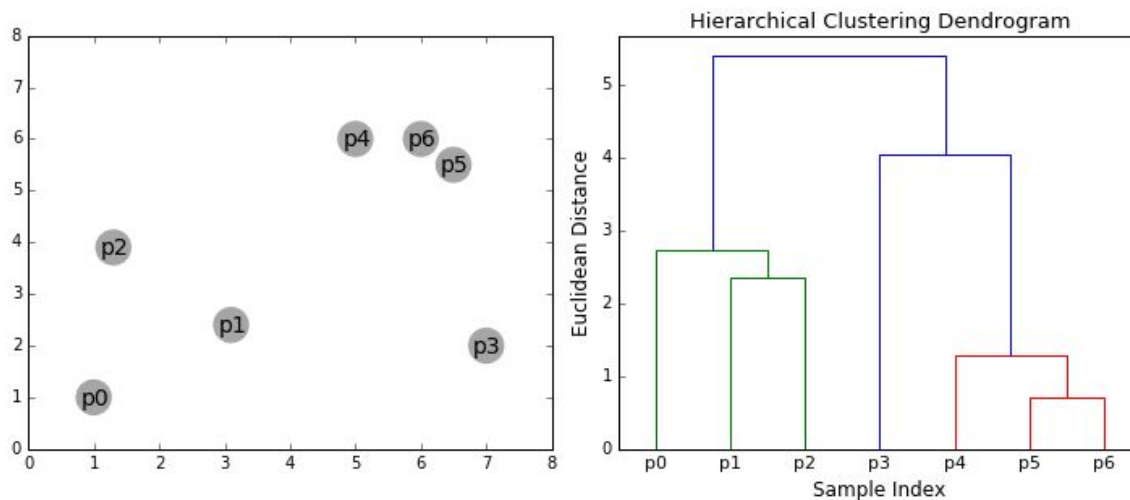


Nested Clusters

Hierarchical Clustering

Agglomerative (Bottom-up) Clustering Algorithm

Most popular hierarchical clustering technique



Procedure

Compute the distance matrix
Let each data point be a cluster

Repeat

Merge the two closest clusters

Update the proximity matrix

Until only a single cluster remains

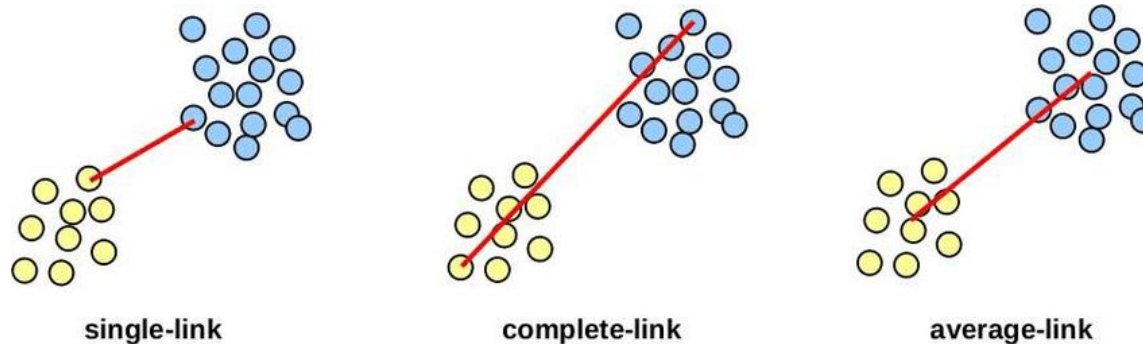
Key operation is the computation of the distance of two clusters

- *Different approaches to defining the distance between clusters distinguish the different algorithms*

Hierarchical Clustering

Agglomerative (Bottom-up) Clustering Algorithm

Important! Distance Metrics

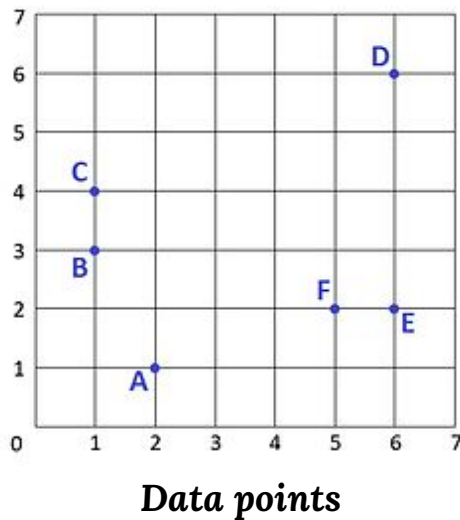


Single link	the distance between 2 clusters is the <u>smallest</u> distance between an element in one cluster and an element in the other
Complete link	the <u>largest</u> distance between an element in one cluster and an element in the other
Average link	the <u>average</u> distance between each element in one cluster and each element in the other

Hierarchical Clustering

Agglomerative Clustering Algorithm - Procedure

Let's perform agglomerative hierarchical clustering on the following data points.



Procedure

Compute the distance matrix
Let each data point be a cluster

Repeat

Merge the two closest clusters

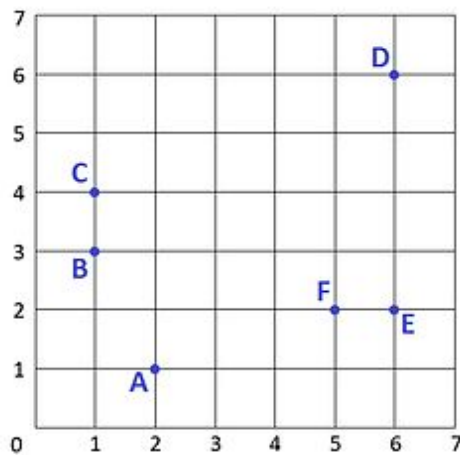
Update the proximity matrix

Until only a single cluster remains

Hierarchical Clustering

Agglomerative Clustering Algorithm - Procedure

First, let's fill matrix of distances between all data points.
For simplicity of calculations, we will use Manhattan distance measure.
Remember, that in agglomerative clustering we assign each observation to its own cluster



Data points

	A	B	C	D	E	F
A	x	3	4	9	5	4
B		x	1	8	6	5
C			x	7	7	6
D				x	4	5
E					x	1
F						x

Distance Matrix

Procedure

Compute the distance matrix

Let each data point be a cluster

Repeat

Merge the two closest clusters

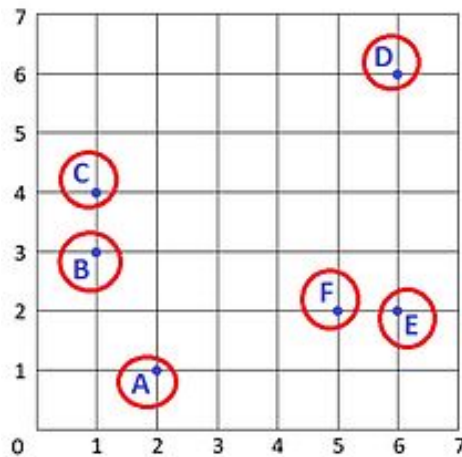
Update the proximity matrix

Until only a single cluster remains

Hierarchical Clustering

Agglomerative Clustering Algorithm - Procedure

Remember, that in agglomerative clustering we assign each observation to its own cluster.



Data points

	A	B	C	D	E	F
A	x	3	4	9	5	4
B		x	1	8	6	5
C			x	7	7	6
D				x	4	5
E					x	1
F						x

Distance Matrix

Procedure

Compute the distance matrix
Let each data point be a cluster

Repeat

Merge the two closest clusters

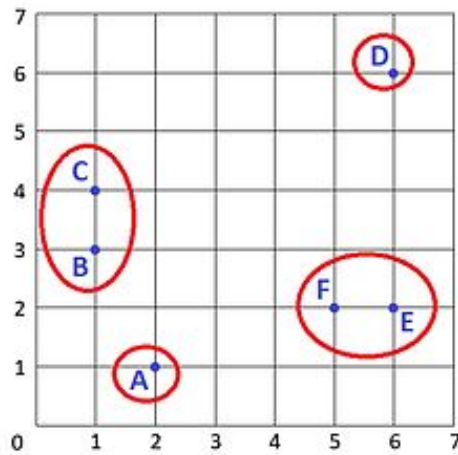
Update the proximity matrix

Until only a single cluster remains

Hierarchical Clustering

Agglomerative Clustering Algorithm - Procedure

The lower half of the distance matrix is not filled as this matrix is symmetric. As you can see clusters E and F are the most similar clusters at this point as well as clusters B and C. We can join them in any order. Let's begin with E and F, we join them.



Data points

	A	B, C	D	E, F
A	x	3	9	4
B, C		x	7	5
D			x	4
E, F				x

Distance Matrix
(after first cluster
union)

Procedure

Compute the distance matrix
Let each data point be a cluster

Repeat

**Merge the two closest
clusters**

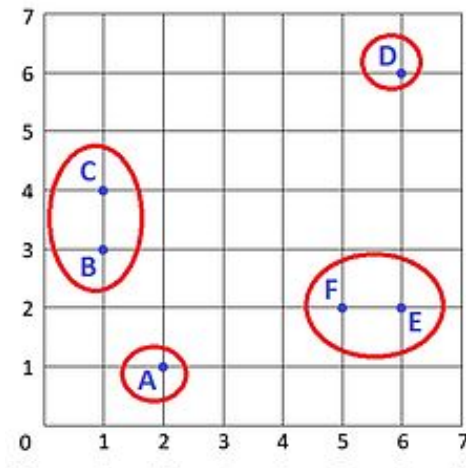
**Update the proximity
matrix**

Until only a single cluster remains

Hierarchical Clustering

Agglomerative Clustering Algorithm - Procedure

The steps 2 and 3 of the algorithm are repeated until we have just one class
(In this example, we use **single link**)



Data points

	A	B, C	D	E, F
A	x	3	9	4
B, C		x	7	5
D			x	4
E, F				x

Distance Matrix
(after first cluster
union)

Procedure

Compute the distance matrix
Let each data point be a cluster

Repeat

Merge the two closest
clusters

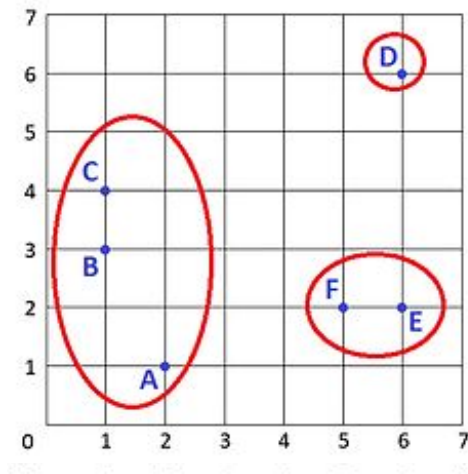
Update the proximity matrix

Until only a single cluster remains

Hierarchical Clustering

Agglomerative Clustering Algorithm - Procedure

The steps 2 and 3 of the algorithm are repeated until we have just one class



Data points

	A, B, C	D	E, F
A, B, C	x	7	4
D		x	4
E, F			x

Distance Matrix

Procedure

Compute the distance matrix
Let each data point be a cluster

Repeat

Merge the two closest clusters

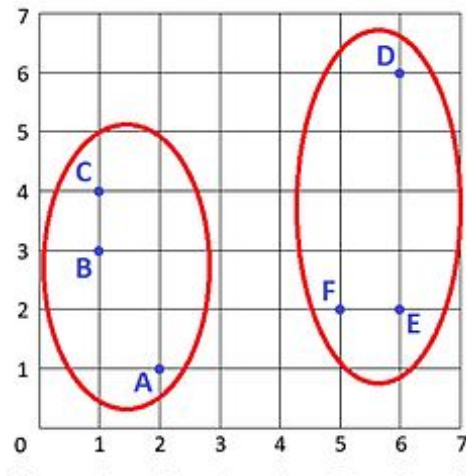
Update the proximity matrix

Until only a single cluster remains

Hierarchical Clustering

Agglomerative Clustering Algorithm - Procedure

The steps 2 and 3 of the algorithm are repeated until we have just one class



Data points

	A, B, C	D, E, F
A, B, C	x	4
D, E, F		x

Distance Matrix

Procedure

Compute the distance matrix
Let each data point be a cluster

Repeat

Merge the two closest clusters

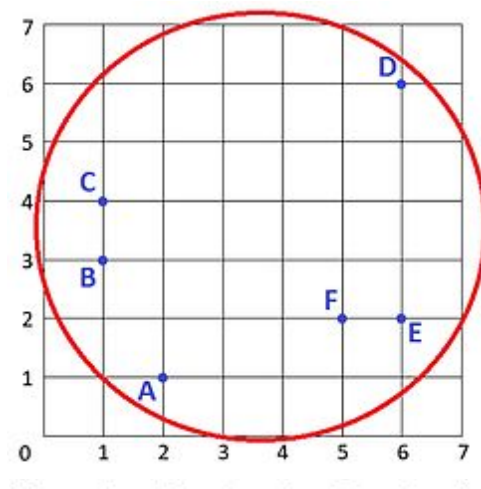
Update the proximity matrix

Until only a single cluster remains

Hierarchical Clustering

Agglomerative Clustering Algorithm - Procedure

Become a single cluster remains



Data points

	A, B, C, D, E, F
A, B, C, D, E, F	x

**Distance
Matrix**

Procedure

Compute the distance matrix
Let each data point be a cluster

Repeat

Merge the two closest
clusters

Update the proximity matrix

**Until only a single cluster
remains**

Hierarchical Clustering

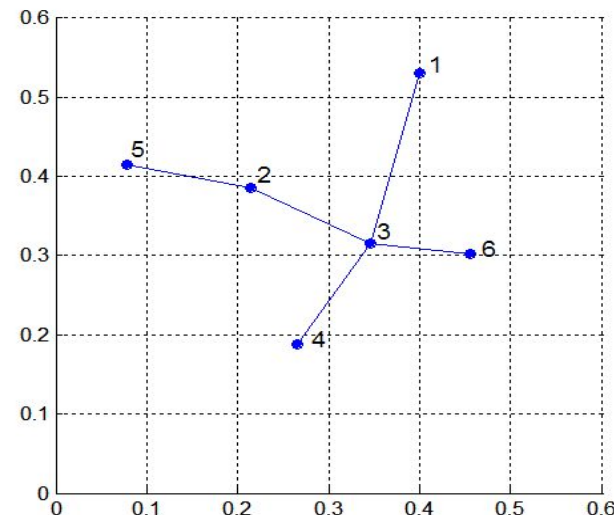
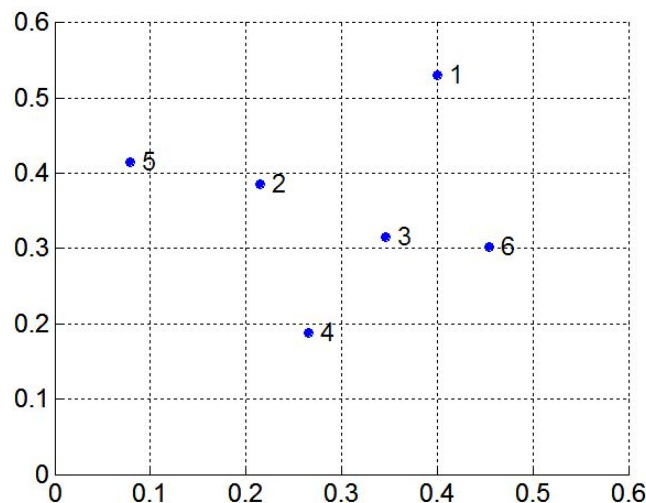
Divisive Hierarchical Clustering

Kruskal's algorithm

Build MST (Minimum Spanning Tree from distance matrix)

MST connects all the points (nodes) together, without any cycles and with the minimum possible total edge weight (distance)

- pre-compute distance matrix
- iteratively connect data points with lowest weight (nearest) if it does not form a cycle



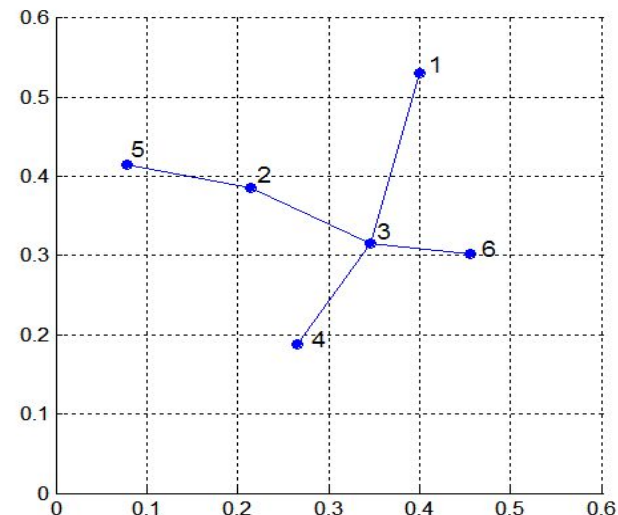
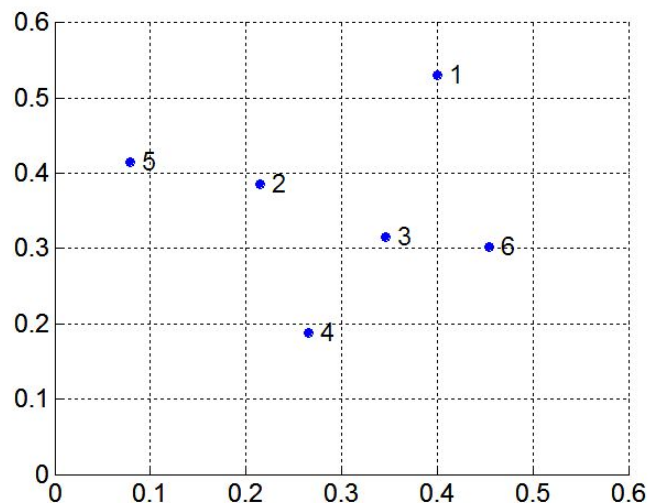
Hierarchical Clustering

Divisive Hierarchical Clustering

Use MST for constructing hierarchy of clusters

Algorithm 7.5 MST Divisive Hierarchical Clustering Algorithm

- 1: Compute a minimum spanning tree for the proximity graph.
 - 2: **repeat**
 - 3: Create a new cluster by breaking the link corresponding to the largest distance (smallest similarity).
 - 4: **until** Only singleton clusters remain
-



Hierarchical Clustering

Time and Space requirements

$O(N^2)$ **space** since it uses the distance matrix

- N is the number of points. (It requires storing the distance matrix)

$O(N^3)$ **time** in many cases

- There are N steps and at each step the size, N^2 , distance matrix must be updated and searched
- Complexity can be reduced to $O(N^2 \log(N))$ time with some approaches by using appropriate data structures

Hierarchical Clustering

Problems and Limitations

- *Once a decision is made to combine two clusters, it cannot be undone*
- *No global objective function is directly minimized*
- *Different schemes have problems with one or more of the following:*
 - *Sensitivity to noise and outliers*
 - *Difficulty handling clusters of different sizes and non-globular shapes*
 - *Breaking large clusters*

Lecture outline

1. General introduction
2. **k-Means clustering**
3. Visualisation of clustering tendency (VAT)
4. **Hierarchical clustering (Part 2)**
5. **Dimensionality reduction (Part 2)**

Motivation: High-dimensional data

- High-dimensional data problem

Many features $\neq$ useful information

- Irrelevant features can confuse models
- Distances become less meaningful

→ Curse of dimensionality

Dimensionality reduction

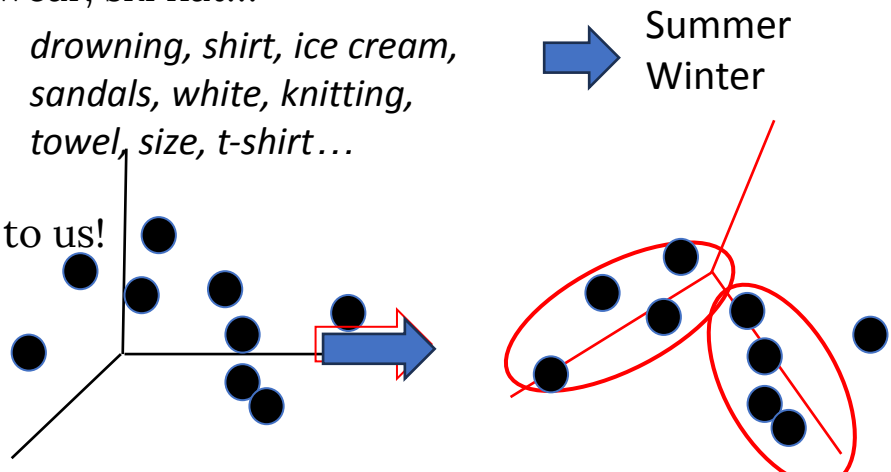
Goal: Reduce features ($N \rightarrow n$, $n \ll N$)

- Keep useful information
- Remove irrelevant features
- Create new features (not just select)
- Often requires domain knowledge

Intuition

- Mathematical technique that automatically identifies the most useful independent aspects of the data
 - Each aspect is a '**semantic primitive**' in terms of which objects in the world are understood.
- Example: Input texts about *summer/winter*, all kinds of words mixed
 - What dimensionality reduction does:
groups words, with strengths (+ and -) into categories:
 - Activities: swimming, beach, skiing, campfire...
 - Food: ice cream, watermelon, fondue, hot chocolate...
 - Clothing: t-shirt, sandals, swimwear, ski hat...
 - The dimensions are:
 - Orthonormal
 - Sorted by strength
 - The dimensions may not be sensible to us!

$$z = 0.8(\text{beach}) + 0.7(\text{swimming}) + 0.6(\text{sandals}) - 0.8(\text{skiing}) - 0.7(\text{snow}) \dots$$



Dimension reduction

Input: A dataset with N features and K objects

Output: A transformed dataset with the same K objects now represented using $n \ll N$ features

- n is often 2 or 3, so that you can easily visualize the data

Object ID	Feature 1	Feature 2	Feature 3	...	Feature N
item1					
item2					
itemK					



Object ID	New Feature 1	New Feature 2
item1		
item2		
itemK		

e.g., for $n=2$:

Transforming from N to $n \ll N$ dimensions

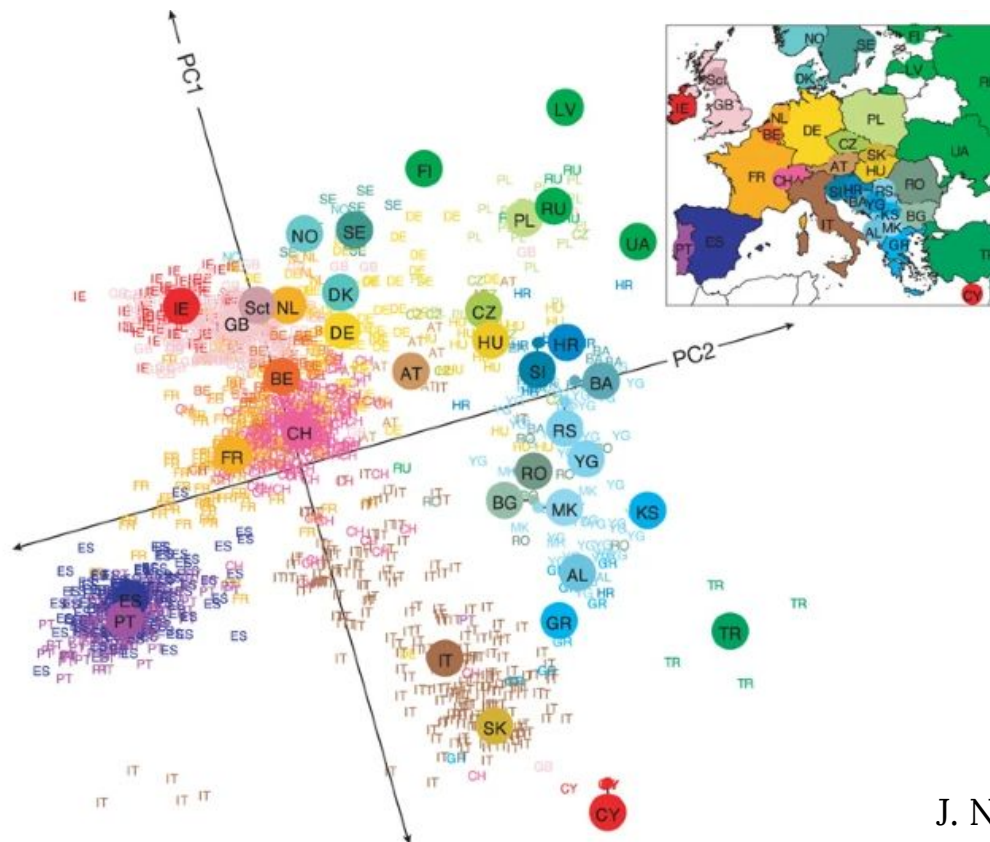
- You are **transforming your data points into a new space**
- The transformation should preserve the characteristics of the data, i.e. distances between pairs of objects
 - If **a pair of objects is close** before the transformation, they should still be close after the transformation
 - If **a pair of objects is far** apart before the transformation, they should still be far apart after the transformation
 - The **set of nearest neighbours of an object** before the transformation should ideally be the same after the transformation

How do I find useful new features?

Genes mirror geography within Europe

Which countries resemble each other?
Why?

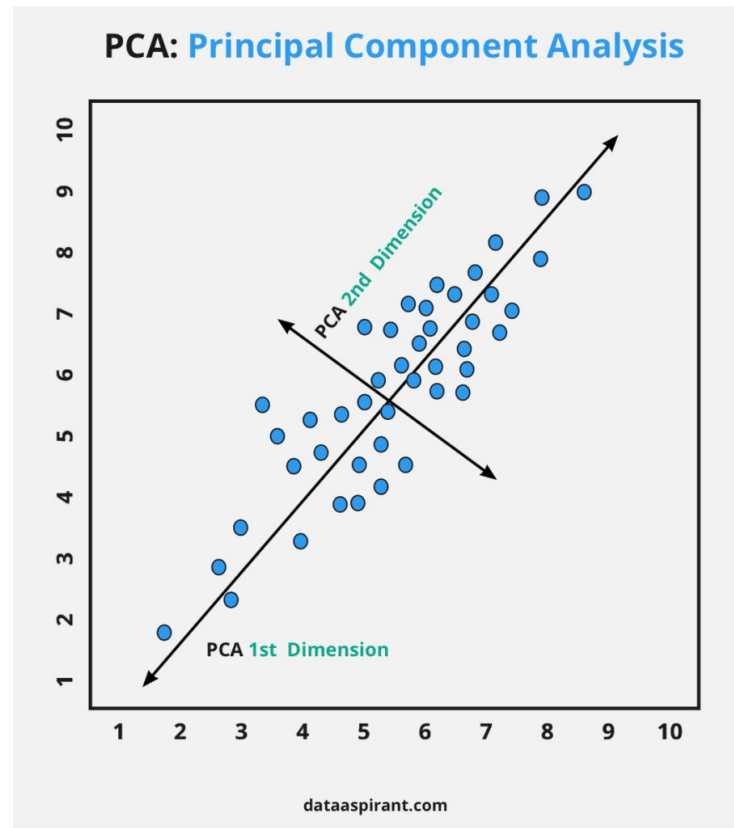
researchers studied the genomes of 3,192 European individuals at 500,568 loci (positions across the genome)



J. Novembre et al., *Nature* 000, 1-4 (2008)

Principal components analysis (PCA)

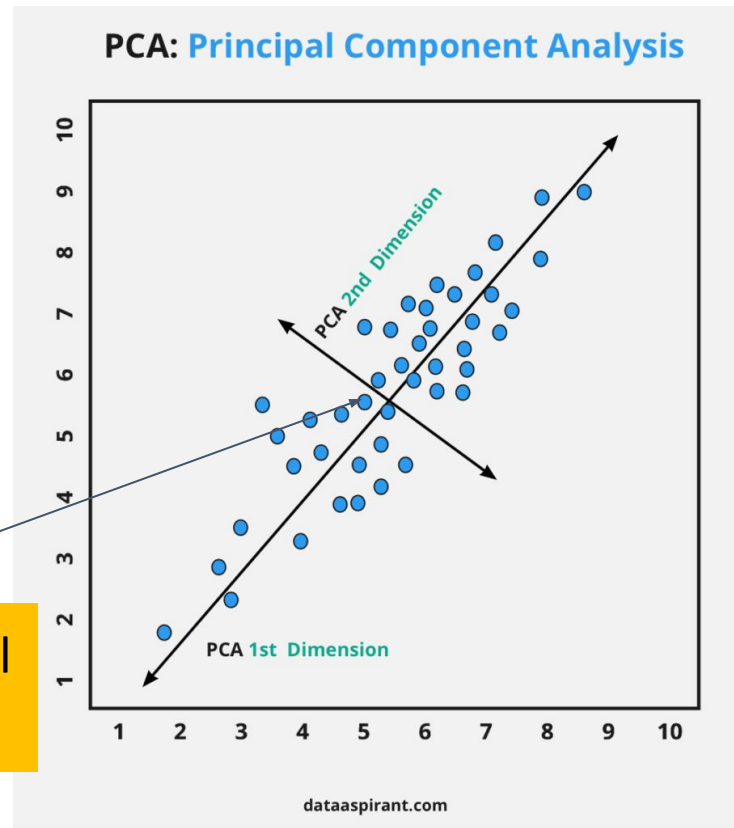
- Mathematical technique that **automatically** identifies the **most useful independent** aspects of the data



directions
(principal
components)
capture the
largest variation
in the data

Principal components analysis (PCA)

- Mathematical technique that **automatically** identifies the **most useful independent** aspects of the data



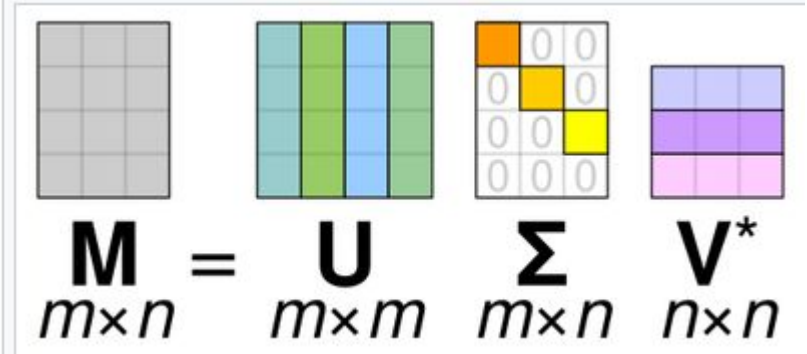
directions
(principal
components)
capture the
largest variation
in the data

Principal components analysis (PCA)

- Mathematical technique that **automatically identifies the most useful independent** aspects of the data

Related approaches:

- Singular Value Decomposition (SVD)** Used by engineers to determine essential elements in complex data problems (here called SVD)


$$\begin{matrix} \text{4x6} & \text{4x4} & \text{4x6} & \text{4x4} \\ \mathbf{M} & = & \mathbf{U} & \mathbf{\Sigma} & \mathbf{V}^* \\ m \times n & & m \times m & m \times n & n \times n \end{matrix}$$

$$n \ll m$$

sparse term-document matrix $\rightarrow$ dense representations

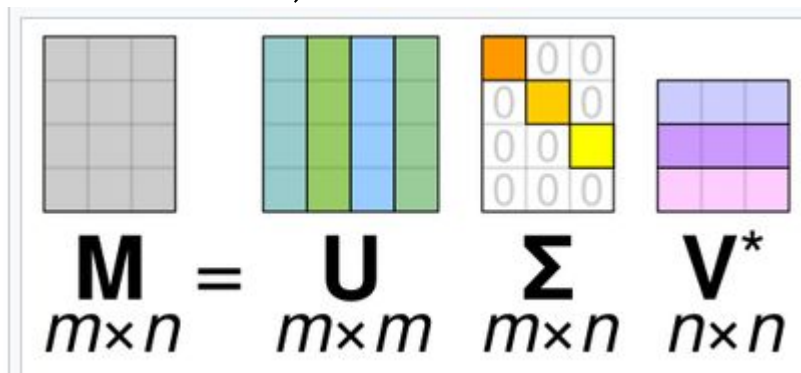
Principal components analysis (PCA)

- Mathematical technique that **automatically identifies the most useful independent** aspects of the data

Nice paper!

Related approaches:

- Singular Value Decomposition (SVD)** determine essential elements in comp (called SVD)



Neural Word Embedding as Implicit Matrix Factorization

Omer Levy
Department of Computer Science
Bar-Ilan University
omerlevy@gmail.com

Yoav Goldberg
Department of Computer Science
Bar-Ilan University
yoav.goldberg@gmail.com

Abstract

We analyze skip-gram with negative-sampling (SGNS), a word embedding method introduced by Mikolov et al., and show that it is implicitly factorizing a word-context matrix, whose cells are the pointwise mutual information (PMI) of the respective word and context pairs, shifted by a global constant. We find that another embedding method, NCE, is implicitly factorizing a similar matrix, where each cell is the (shifted) log conditional probability of a word given its context. We show that using a sparse *Shifted Positive PMI* word-context matrix to represent words improves results on two word similarity tasks and one of two analogy tasks. When dense low-dimensional vectors are preferred, exact factorization with SVD can achieve solutions that are at least as good as SGNS's solutions for word similarity tasks. On analogy questions SGNS remains superior to SVD. We conjecture that this stems from the weighted nature of SGNS's factorization.

sparse term-document matrix → dense representations

Principal components analysis (PCA)

- Mathematical technique that **automatically identifies the most useful independent aspects of the data**

Related approaches:

- **Latent Semantic Analysis (LSA)** is used to analyse the relationship between terms and documents (e.g., topic analysis), document classification/clustering. Is also based on SVD and also finds a low-rank approximation to the term-document matrix.

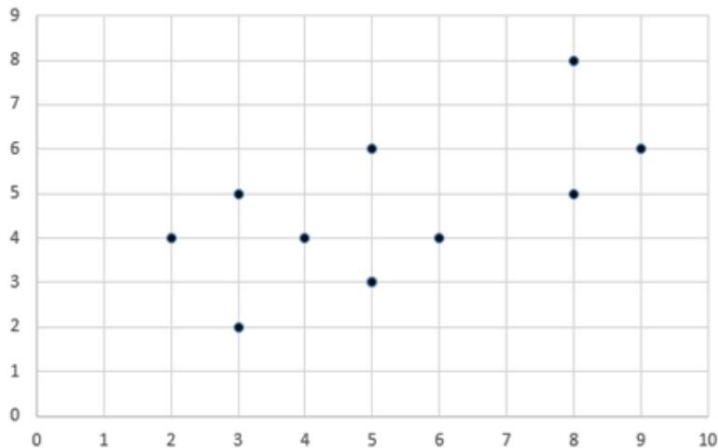
Is based on the Distributional
(Semantic) Hypothesis:
meaning $\approx$ context

Details of PCA

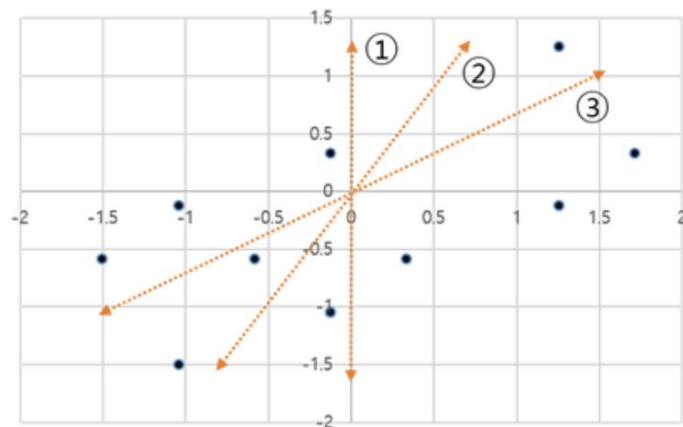
- **Find a new set of features** that together **better capture the variability of the data**
 - Each dimension is a (linear) weighted combination of features
 - The dimensions are orthogonal (= **independent**)
 - Each dimension has an '**importance**' strength
 - The dimensions are ordered by **strength**:
 - **First dimension** chosen to **capture as much of the variability as possible**
 - The **second dimension** is **orthogonal to the first**, and subject to that constraint, **captures as much of the remaining variability** as possible
 - The **third dimension** is orthogonal to the first and second, and subject to that constraint, captures as much of the remaining variability as possible
 - etc.
- A good visualisation:
<http://setosa.io/ev/principal-component-analysis>

Details of PCA - drawing

- The dimensions are ordered by strength:
 - **First dimension** chosen to **capture as much of the variance as possible**
 - The second dimension is orthogonal to the first, and subject to that constraint, captures as much of the remaining variance as possible

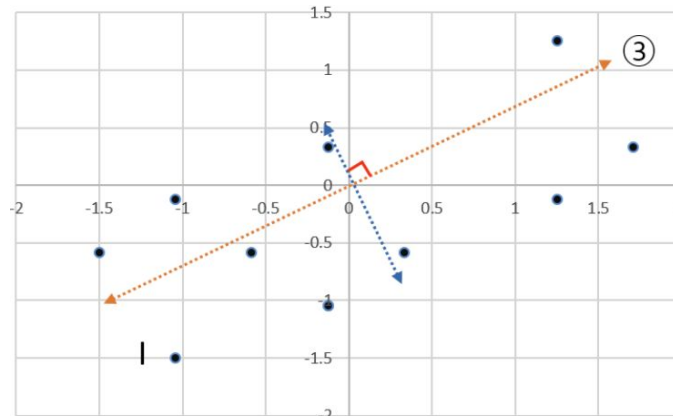
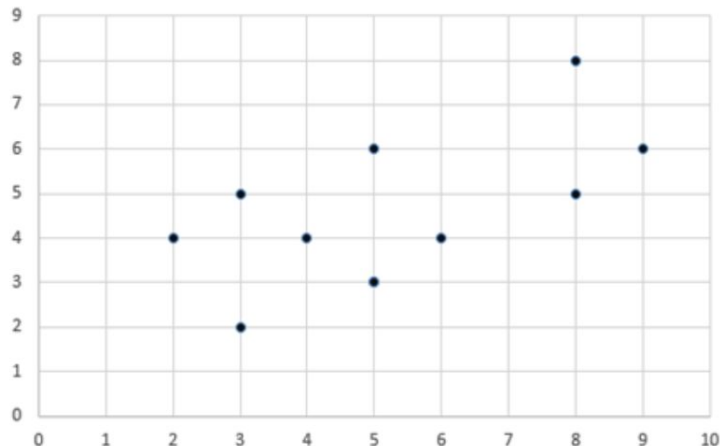


x1	5	9	5	4	8	3	2	3	8	6
x2	6	6	3	4	8	2	4	5	5	4



Details of PCA - drawing

- The dimensions are ordered by strength:
 - **First dimension** chosen to **capture as much of the variance as possible**
 - The **second dimension** is **orthogonal to the first**, and **subject to that constraint, captures as much of the remaining variance as possible**

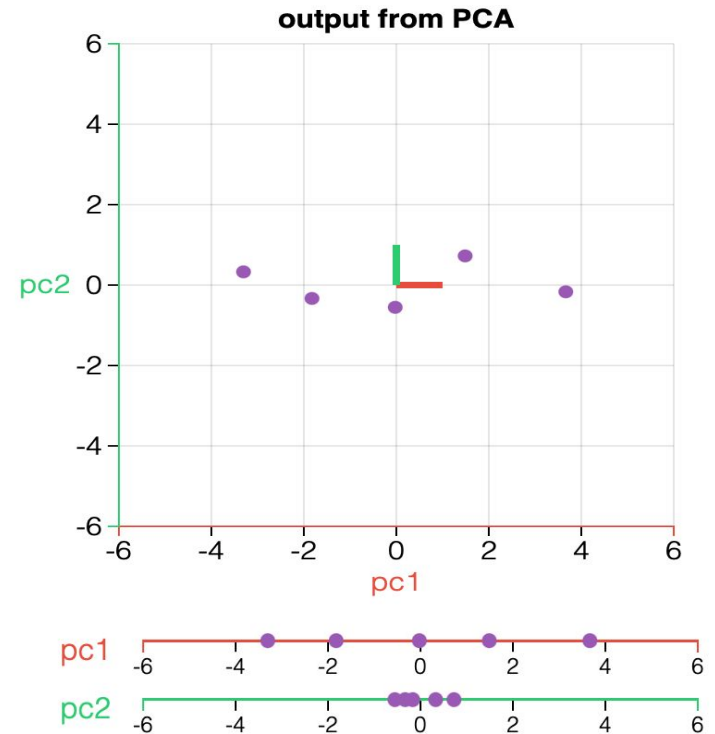
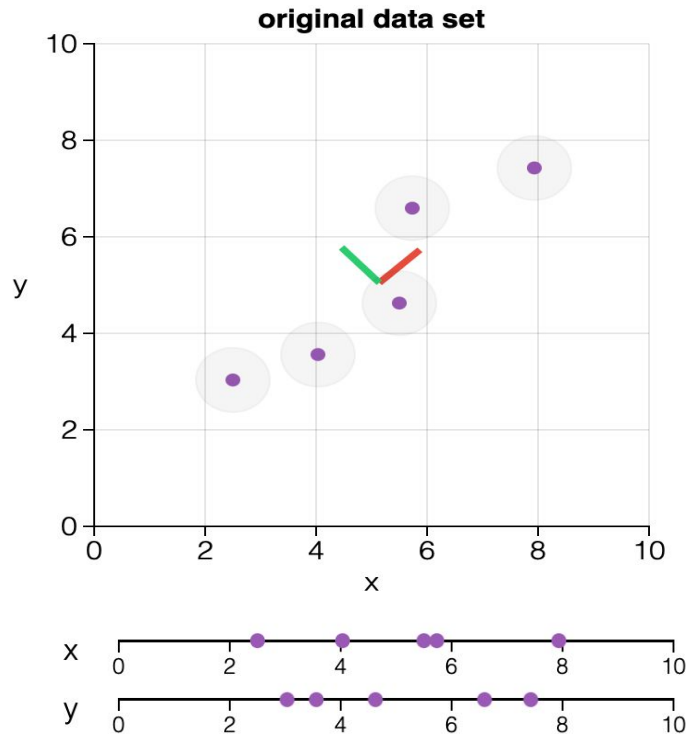


x1	5	9	5	4	8	3	2	3	8	6
x2	6	6	3	4	8	2	4	5	5	4

PCA example

Find a new set of features that better captures the variance of the data

<https://setosa.io/ev/principal-component-analysis/>



New dimensions,

Principal components analysis in Python

`sklearn.decomposition`

Will practice in workshop

PCA: Good aspects

- May help to eliminate irrelevant features or reduce noise
 - Fewer features → smaller machine learning models → faster answer
 - Better feature set by transformation → more accurate answer
- Avoid curse of dimensionality
- Allow data to be more easily visualised
- Reduce amount of time and memory required by algorithms
- Principal components are independent

PCA limitations

- It is not clear how many dimensions to consider
- It may be non-intuitive to interpret the principal components
- There may be some information loss
- Assumes only a linear combinations of the features. And since the model is not probabilistic (the scores come from the matrix manipulation), you can't mathematically combine its results with probability-based results of other procedures
- The PCA reduced dimensions may not help with classification
- PCA is sensitive to scale (standardize features!)
- PCA is sensitive to outliers

Going beyond PCA

- There are many other methods for reducing dimensionality
 - Canonical Correlation Analysis (CCA), Linear Discriminant Analysis (LDA), etc.
- Which one to use is a matter of taste, compute power, data, and what you want to do

Week 5 Lecture & Workshop 6 Alignment

- **Explore clustering techniques:**
 - k-means algorithm and how it works
 - Choosing the number of clusters (Elbow method, VAT)
 - Understanding when k-means works well (and when it doesn't)
- **Introduce hierarchical clustering:**
 - Agglomerative vs divisive clustering
 - Step-by-step clustering process
 - Interpreting cluster structures
- **Analyse and evaluate clustering results:**
 - Compare different clustering outputs
 - Understand limitations of clustering methods
- **Introduce dimensionality reduction:**
 - PCA intuition and principal components
 - Explained variance ratio
- **Apply PCA for visualisation:**
 - Reduce high-dimensional data
 - Visualise clustering results more clearly

Questions?